Booleans and Conditionals

Course: Kaggle Learn · Instructor: Colin Morris · Lesson: Booleans and Conditionals

1. Introduction to Booleans

In Python, a Boolean is a data type that represents one of two possible values: `True` or `False`. The Boolean type is called `bool`.

Booleans are useful for representing conditions, making comparisons, checking whether something is true, and controlling which parts of a program execute.

Example:

x = Trueprint(x)print(type(x))

Output:

True

<class 'bool'>

Explanation:

- `x = True` assigns the Boolean value `True` to the variable `x`.
- `print(x)` displays the value stored in `x`.
- `type(x)` returns the data type of `x`, which is `bool`.

Python commonly generates Boolean values using comparison operators and Boolean operators, rather than assigning `True` or `False` directly.

2. Comparison Operations

Comparison operators compare two values and return a Boolean result: either `True` or `False`.

2.1 Comparison Operators Reference

Operator	Description	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`<`	Less than	`3 < 5`	`True`
`>`	Greater than	`5 > 3`	`True`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal to	`5 >= 3`	`True`

Example:

print(10 == 10)print(10 != 5)print(3 < 7)print(8 > 12)print(5 <= 5)print(6 >= 9)

Output:

True

True

True

False

True

False

Explanation:

Each comparison evaluates whether the stated relationship between the two values is correct.

2.2 Using Comparisons in Functions

Comparison operators are especially useful inside functions to make decisions based on input values.

Consider a function that checks whether someone meets the minimum age requirement to run for president in the United States.

def can_run_for_president(age): """Can someone of the given age run for president in the US?""" # The US Constitution requires candidates to be at least 35 years old. return age >= 35print("Can a 19-year-old run for president?", can_run_for_president(19))print("Can a 45-year-old run for president?", can_run_for_president(45))

Output:

Can a 19-year-old run for president? False

Can a 45-year-old run for president? True

Explanation:

- `age >= 35` checks whether the supplied age is at least 35.
- `return` sends the resulting Boolean value back to the caller.
- When `age` is `19`, the comparison evaluates to `False`.
- When `age` is `45`, the comparison evaluates to `True`.

The example illustrates Boolean comparisons; actual presidential eligibility also depends on other constitutional requirements.

2.3 Comparing Different Data Types

Comparisons generally work as expected when the values have compatible types, but values that look similar may not be equal.

Example:

print(3.0 == 3)print('3' == 3)

Output:

True

False

Explanation:

- `3.0` is a floating-point number, while `3` is an integer. Python considers them numerically equal.

- `'3'` is a string containing the character `3`, whereas `3` is an integer.
- Python does not automatically treat a numeric string as the equivalent integer during equality comparison.

Important distinction: A value's appearance does not determine its data type.

2.4 Combining Comparisons with Arithmetic Operators

Comparison operators can be combined with arithmetic operators to test mathematical conditions.

One example is checking whether a number is odd using the modulus operator `%`.

The modulus operator returns the remainder after division.

def is_odd(n): return (n % 2) == 1print("Is 100 odd?", is_odd(100))print("Is -1 odd?", is_odd(-1))

Output:

Is 100 odd? False

Is -1 odd? True

Explanation:

- `n % 2` calculates the remainder after dividing `n` by `2`.
- `(n % 2) == 1` checks whether that remainder is `1`.
- In Python, `100 % 2` is `0`, so the function returns `False`.
- `-1 % 2` is `1` in Python, so the function returns `True`.

This implementation works for positive and negative integers because Python's modulo operation gives a non-negative remainder when the divisor is positive.

2.5 Difference Between `=` and `==`

The assignment operator `=` and the equality operator `==` serve different purposes.

Operator	Purpose	Example
`=`	Assigns a value to a variable	`n = 2`

`==`	Checks whether two values are equal	`n == 2`

Example:

n = 2 # Assigns 2 to nprint(n == 2) # Checks whether n equals 2

Output:

True

Remember:

- `n = 2` changes the value stored in `n`.
- `n == 2` evaluates to `True` if `n` equals `2`, and `False` otherwise.

Using `=` instead of `==` in a comparison condition causes an error because ordinary assignment is not an expression that can be used as an `if` condition in Python.

3. Combining Boolean Values

Python provides three logical operators for combining or modifying Boolean values:

- `and`
- `or`
- `not`

These operators allow programs to evaluate more complex conditions.

3.1 The `and` Operator

The `and` operator returns `True` only when both conditions are true.

A	B	`A and B`
`True`	`True`	`True`

`True`	`False`	`False`
`False`	`True`	`False`
`False`	`False`	`False`

Example:

print(True and True)print(True and False)print(False and True)print(False and False)

Output:

True

False

False

False

3.2 The `or` Operator

The `or` operator returns `True` when at least one condition is true. It returns `False` only when both conditions are false.

A	B	`A or B`
`True`	`True`	`True`
`True`	`False`	`True`
`False`	`True`	`True`

False	False	False

Example:

print(True or True)print(True or False)print(False or True)print(False or False)

Output:

True

True

True

False

3.3 The `not` Operator

The `not` operator reverses a Boolean value.

A	not A
True	False
False	True

Example:

print(not True)print(not False)

Output:

False

True

3.4 Combining Boolean Operators in a Function

The `and` operator can combine multiple requirements into a single condition.

For example, consider the age and citizenship requirements used in the earlier presidential eligibility illustration.

def can_run_for_president(age, is_natural_born_citizen): """Check the age and citizenship conditions.""" # Both conditions must be satisfied. return is_natural_born_citizen and (age >= 35)print(can_run_for_president(19, True))print(can_run_for_president(55, False))print(can_run_for_president(55, True))

Output:

False

False

True

Explanation:

- The first argument provides the person's age.
- The second argument provides a Boolean citizenship-status value.
- `is_natural_born_citizen and (age >= 35)` requires both conditions to be true.
- A person who meets only one of the two conditions receives `False`.
- When both conditions are true, the function returns `True`.

This is a simplified illustration of two conditions, not a complete legal eligibility test.

3.5 Order of Operations and Operator Precedence

When an expression contains multiple logical operators, Python follows a defined order of precedence.

For these operators, the order from highest to lowest precedence is:

1. `not`
2. `and`
3. `or`

Example:

print(True or True and False)

Output:

True

Explanation:

Python evaluates `and` before `or`, so the expression is equivalent to:

print(True or (True and False))

Evaluation:

1. `True and False` evaluates to `False`.
2. `True or False` evaluates to `True`.

Therefore, the final result is `True`.

If the expression were evaluated strictly from left to right, the result could be different. Python's operator precedence determines the actual evaluation order.

Best practice: Use parentheses to make your intended order explicit, even when you already know the precedence rules.

3.6 Using Parentheses to Improve Readability

Consider a condition that determines whether someone is prepared for the weather.

prepared_for_weather = (have_umbrella or (rain_level < 5 and have_hood) or not (rain_level > 0 and is_workday))

This expression represents three alternative ways of being prepared:

1. Having an umbrella.
2. Having a hood when the rain level is below `5`.
3. Not having the combination of rain and a workday.

Explanation:

- `have_umbrella` is checked first as an alternative condition.
- `(rain_level < 5 and have_hood)` requires both light enough rain and a hood.
- `not (rain_level > 0 and is_workday)` reverses the result of the combined rain-and-workday condition.
- The outer `or` operators mean that any one of these alternatives can make the overall result true.

The original expression without parentheses is harder to read and can produce behavior that does not match the programmer's intended logic.

Parentheses can also be used to make individual comparisons clearer:

prepared_for_weather = (have_umbrella or ((rain_level < 5) and have_hood) or (not (rain_level > 0 and is_workday)))

Both versions have the same logical meaning. The additional parentheses emphasize the structure.

4. Conditional Statements

Conditional statements allow a program to execute different blocks of code depending on whether a condition is true or false.

Python uses three main conditional keywords:

- `if`
- `elif`
- `else`

4.1 The `if` Statement

The `if` statement executes its indented block only when its condition evaluates to `True`.

Syntax:

if condition: # Code executes when condition is True

Example:

x = 5if x > 0: print("x is positive")

Output:

x is positive

Explanation:

- Python evaluates `x > 0`.
- Since `5 > 0` is `True`, the indented statement executes.
- If the condition were false, the block would be skipped.

4.2 The `elif` Statement

`elif` is short for else if. It checks another condition when the preceding `if` or `elif` condition has not been satisfied.

You can include as many `elif` statements as needed.

4.3 The `else` Statement

The `else` block executes when none of the preceding `if` or `elif` conditions is true.

Both `elif` and `else` are optional.

4.4 Using `if`, `elif`, and `else` Together

The following function classifies a number as zero, positive, or negative.

def inspect(x): if x == 0: print(x, "is zero") elif x > 0: print(x, "is positive") elif x < 0: print(x, "is negative") else: print(x, "is unlike anything I've ever seen...")inspect(0)inspect(-15)

Output:

0 is zero

-15 is negative

Explanation:

- `if x == 0` checks whether the number is zero.
- `elif x > 0` checks whether it is positive if the first condition was false.
- `elif x < 0` checks whether it is negative.
- `else` handles any remaining case.

For ordinary numeric inputs, the first three conditions cover zero, positive values, and negative values. The final `else` is effectively unreachable for ordinary integers and floats, but it demonstrates the fallback structure.

Python executes only the first matching branch in an `if`/`elif`/`else` chain.

4.5 Colons and Indentation

Python uses colons (`:`) and indentation to define code blocks.

Example:

def f(x): if x > 0: print("Only printed when x is positive; x =", x) print("Also only printed when x is positive; x =", x) print("Always printed, regardless of x's value; x =", x)f(1)f(0)

Output:

Only printed when x is positive; x = 1

Also only printed when x is positive; x = 1

Always printed, regardless of x's value; x = 1

Always printed, regardless of x's value; x = 0

Explanation:

- `def f(x):` begins a function definition.
- `if x > 0:` begins a conditional block.
- The two `print()` statements indented inside the `if` block execute only when `x > 0`.
- The final `print()` statement is indented inside the function but outside the `if` block. Therefore, it executes every time the function is called.

Python conventionally uses four spaces per indentation level.

Important rule: Indentation is part of Python syntax, not merely a formatting preference.

5. Boolean Conversion with `bool()`

Python provides the built-in `bool()` function to convert values into Boolean values.

Just as `int()` converts suitable values to integers and `float()` converts suitable values to floating-point numbers, `bool()` determines the truth value of an object.

5.1 Converting Numbers and Strings

Example:

print(bool(1)) # Non-zero numbers are truthyprint(bool(0)) # Zero is falsyprint(bool("asf")) # Non-empty strings are truthyprint(bool("")) # Empty strings are falsy

Output:

True

False

True

False

Explanation:

- Non-zero numeric values are generally considered truthy.
- Numeric zero is falsy.
- Non-empty strings are truthy, even if their contents represent something like `"0"`.
- An empty string `""` is falsy.

5.2 Truthy and Falsy Values

Python classifies objects according to their truth value.

Value	Boolean conversion	Classification
`1`	`True`	Truthy
`-1`	`True`	Truthy
`0`	`False`	Falsy
`3.5`	`True`	Truthy
`""`	`False`	Falsy
`"hello"`	`True`	Truthy
`[]`	`False`	Falsy
`[1, 2]`	`True`	Truthy
`()`	`False`	Falsy

`{}`	`False`	Falsy
`None`	`False`	Falsy

Empty sequences and collections, such as strings, lists, tuples, dictionaries, and sets, are falsy. Non-empty ones are generally truthy.

Other important falsy values include `False` itself and numeric zero in its various forms.

5.3 Using Non-Boolean Values in Conditions

Python allows values other than `True` and `False` to appear in conditional statements. Python implicitly evaluates their truth values.

Example:

if 0: print(0)elif "spam": print("spam")

Output:

spam

Explanation:

1. Python evaluates `0` as falsy, so the first block is skipped.
2. Python evaluates `"spam"` as truthy because it is a non-empty string.
3. The `elif` block executes and prints `"spam"`.

This behavior is called truth-value testing. It is often used to check whether a collection or string contains something without explicitly comparing it to `True`.

For example:

items = [1, 2, 3]if items: print("The list is not empty")

Output:

The list is not empty

Because `items` is non-empty, its truth value is `True`.

6. Important Differences and Comparisons

6.1 Comparison Operators vs. Logical Operators

Feature	Comparison operators	Logical operators
Operators	`==`, `!=`, `<`, `>`, `<=`, `>=`	`and`, `or`, `not`
Purpose	Compare values	Combine or reverse conditions
Example	`5 > 3`	`(5 > 3) and (2 < 4)`
Result	A Boolean value	A truth value; with Boolean operands, a Boolean result

6.2 `and` vs. `or` vs. `not`

Operator	Meaning	When the result is true
`and`	Both conditions must be true	Both operands are truthy
`or`	At least one condition must be true	At least one operand is truthy
`not`	Reverses a truth value	The operand is falsy

6.3 `if` vs. `elif` vs. `else`

Keyword	Purpose	When it executes

`if`	Starts a conditional chain	When its condition is true
`elif`	Checks an additional condition	When earlier conditions were false and its condition is true
`else`	Provides a fallback block	When all preceding conditions are false

6.4 Truthy vs. Falsy Values

Truthy examples	Falsy examples
`1`, `-5`, `3.14`	`0`, `0.0`
`"hello"`, `"0"`	`""`
`[1, 2]`	`[]`
`(1,)`	`()`
`{"a": 1}`	`{}`
`True`	`False`
Most other non-empty objects	`None`

7. Practical Examples

7.1 Check Whether a Number Is Positive, Negative, or Zero

def check_number(n): if n > 0: return "Positive" elif n < 0: return "Negative" else: return "Zero"print(check_number(10))print(check_number(-4))print(check_number(0))

Output:

Positive

Negative

Zero

Explanation:

The function checks the number against zero. The `if`/`elif`/`else` chain ensures that exactly one result is returned for each ordinary numeric input.

7.2 Check Whether a Number Is Even and Within a Range

def check_number(n): return (n % 2 == 0) and (n >= 1 and n <= 100)print(check_number(20))print(check_number(101))print(check_number(7))

Output:

True

False

False

Explanation:

The function requires two things:

1. The number must be even: `n % 2 == 0`.
2. The number must be between `1` and `100`, inclusive.

The `and` operator returns `True` only when both requirements are satisfied.

7.3 Check Whether a String Is Empty

def check_string(text): if text: return "String is not empty" else: return "String is empty"print(check_string("Python"))print(check_string(""))

Output:

String is not empty

String is empty

Explanation:

- `"Python"` is a non-empty string and therefore truthy.
- `""` is an empty string and therefore falsy.
- Python implicitly converts each string to its truth value when evaluating the `if` condition.

8. Quick Reference

8.1 Operators and Syntax

Syntax	Purpose	Example
`True`, `False`	Boolean literals	`x = True`
`==`	Equality comparison	`x == 5`
`!=`	Inequality comparison	`x != 5`
`<`, `>`	Less than, greater than	`x < 10`
`<=`, `>=`	Less/greater than or equal to	`x >= 10`
`%`	Modulus; returns remainder	`n % 2`
`and`	Both conditions must hold	`x > 0 and x < 10`

`or`	At least one condition must hold	`x < 0 or x > 10`
`not`	Reverses truth value	`not is_valid`
`if`	Starts a conditional block	`if x > 0:`
`elif`	Tests another condition	`elif x < 0:`
`else`	Executes if no earlier condition matches	`else:`
`bool(x)`	Converts a value to its Boolean truth value	`bool([])` → `False`

8.2 Operator Precedence

From highest to lowest among the operators covered in this lesson:

Precedence	Operator	Meaning
1	`not`	Logical negation
2	`and`	Logical conjunction
3	`or`	Logical disjunction

Use parentheses to make expressions easier to understand and to explicitly control evaluation order.

8.3 Conditional Structure

if condition_1: # Executes if condition_1 is true ...elif condition_2: # Executes if condition_1 is false # and condition_2 is true ...else: # Executes if no earlier condition is true ...

8.4 Truth-Value Rules

- `bool(0)` is `False`.
- `bool("")` is `False`.
- `bool([])` is `False`.
- `bool(None)` is `False`.
- Non-zero numbers are truthy.
- Non-empty strings and collections are truthy.
- Python can evaluate truthy and falsy objects directly in conditional statements.

9. Common Mistakes to Remember

1. Mistake: Using `=` instead of `==` for a comparison.
    - Correction: Use `=` to assign a value and `==` to check equality.
2. Mistake: Assuming `'3' == 3` is `True`.
    - Correction: A string and an integer are different types. Convert the string if numeric comparison is intended.
3. Mistake: Assuming `and` and `or` have the same precedence.
    - Correction: `and` is evaluated before `or`. Use parentheses to make the intended logic clear.
4. Mistake: Thinking `or` requires both conditions to be true.
    - Correction: `or` is true if at least one operand is truthy.
5. Mistake: Forgetting that `not` reverses a truth value.
    - Correction: `not True` is `False`, and `not False` is `True`.
6. Mistake: Using incorrect indentation in conditional blocks.
    - Correction: End conditional headers with a colon (`:`) and indent their bodies consistently, conventionally by four spaces.
7. Mistake: Assuming `else` can have its own condition.
    - Correction: Use `elif` when another condition needs to be checked. `else` has no condition.
8. Mistake: Assuming every statement inside a function belongs to its `if` block.
    - Correction: Indentation determines block membership. A statement outside the `if` block but inside the function executes regardless of that condition.
9. Mistake: Assuming only Boolean values can be used in an `if` statement.
    - Correction: Python accepts other objects and evaluates their truth values implicitly.
10. Mistake: Assuming all non-empty-looking values are truthy without considering their types.

    - Correction: Remember the main falsy values: `False`, `None`, numeric zero, and empty strings or collections.

Lesson takeaway: Booleans represent truth values, comparison operators create Boolean results, logical operators combine conditions, and conditional statements use those conditions to control program execution. Understanding these concepts is essential for writing Python programs that make decisions.